

Содержание

1. Жизненный цикл Java-программы	2
2. Типы данных и операторы в Java	2
3. Основы ООП	4
4. Паттерны Singleton и Factory Method	5
5. Коллекции Java	7
6. Исключения в Java	9
7. Потоки ввода-вывода	10
8. Многопоточность	11
9. Основы Spring Boot	13
10. REST API в Spring Boot	14
11. Dependency Injection	15
12. Работа с базой данных через Spring Data JPA	16
13. Тестирование с JUnit	18
14. Сериализация объектов в JSON	19
15. Основы безопасности	20
16. Асинхронные методы	21
17. Планирование задач	23
18. Конфигурация приложения	24
19. MVC-паттерн	25
20. Swagger	26
21. Аннотации в Java	27
22. Лямбда-выражения и функциональные интерфейсы	28
23. Streams API	29
24. Generics в Java	31
25. Работа с датой и временем	32
26. Паттерн Builder	33
27. Паттерн Observer	35
28. Кэширование и оптимизация	36
29. Транзакции в Spring	37
30. Документирование REST API	38

1. Жизненный цикл Java-программы

Жизненный цикл Java-программы включает несколько этапов: написание исходного кода, компиляцию, загрузку байт-кода, проверку и выполнение программы.

Сначала программист пишет исходный код в файле с расширением .java. Затем компилятор `javac` преобразует этот код в байт-код, который сохраняется в файле с расширением .class. Байт-код не является машинным кодом конкретного процессора, поэтому он может выполняться на разных операционных системах.

После компиляции программа запускается виртуальной машиной Java – JVM. JVM загружает байт-код, проверяет его безопасность и корректность, а затем выполняет программу. За счёт этого Java считается переносимым языком: один и тот же скомпилированный код может работать на Windows, Linux и других системах, если там установлена подходящая среда Java.

JVM (Java Virtual Machine) – виртуальная машина, которая выполняет байт-код Java.

JRE (Java Runtime Environment) – среда выполнения Java. Она включает JVM и стандартные библиотеки, необходимые для запуска программ.

JDK (Java Development Kit) – комплект разработчика. Он включает JRE, компилятор `javac`, отладочные инструменты и другие средства разработки.

Пример жизненного цикла:

Main.java -> `javac Main.java` -> Main.class -> JVM -> выполнение программы

Таким образом, JDK нужен для разработки, JRE – для запуска, а JVM – непосредственно для выполнения байт-кода.

2. Типы данных и операторы в Java

В Java есть примитивные и ссылочные типы данных. Примитивные типы хранят простые значения, а ссылочные типы хранят ссылку на объект.

К примитивным типам относятся:

- `byte` – целое число малого размера;
- `short` – короткое целое число;
- `int` – основной тип для целых чисел;
- `long` – целое число большого размера;
- `float` – число с плавающей точкой одинарной точности;
- `double` – число с плавающей точкой двойной точности;

- `char` – один символ;
- `boolean` – логическое значение `true` или `false`.

Пример объявления переменных:

```
1 int age = 20;  
2 double price = 15.5;  
3 char grade = 'A';  
4 boolean active = true;
```

java

Операторы в Java используются для выполнения действий над переменными и значениями.

Арифметические операторы:

- `+` – сложение;
- `-` – вычитание;
- `*` – умножение;
- `/` – деление;
- `%` – остаток от деления.

Пример:

```
1 int a = 10;  
2 int b = 3;  
3  
4 int sum = a + b;  
5 int remainder = a % b;
```

java

Логические операторы применяются для работы с условиями:

- `&&` – логическое И;
- `||` – логическое ИЛИ;
- `!` – логическое НЕ.

Пример:

```
1 int age = 18;  
2 boolean hasPassport = true;  
3  
4 boolean canRegister = age >= 18 && hasPassport;
```

java

Также часто используются операторы сравнения: ==, !=, >, <, >=, <=.

3. Основы ООП

ООП – объектно-ориентированное программирование. Это подход, при котором программа строится из объектов. Каждый объект объединяет данные и методы для работы с этими данными.

Класс – это шаблон, по которому создаются объекты. Объект – конкретный экземпляр класса.

Пример класса:

```
1  class User {
2      private String name;
3
4      public void setName(String name) {
5          this.name = name;
6      }
7
8      public String getName() {
9          return name;
10     }
11 }
```

java

Основные принципы ООП:

1. Инкапсуляция – скрытие внутреннего состояния объекта. Обычно поля делают private, а доступ к ним дают через методы.
2. Наследование – возможность создавать новый класс на основе существующего.
3. Полиморфизм – возможность использовать разные объекты через общий тип.
4. Абстракция – выделение главных характеристик объекта без лишних деталей.

Пример наследования и полиморфизма:

```
1  class Animal {
2      void sound() {
3          System.out.println("Sound");
```

java

```

4    }
5    }
6
7    class Dog extends Animal {
8        @Override
9        void sound() {
10            System.out.println("Bark");
11        }
12    }
13
14    Animal animal = new Dog();
15    animal.sound();

```

В этом примере переменная имеет тип `Animal`, но фактически хранит объект `Dog`. При вызове метода выполняется переопределённая версия из класса `Dog`.

ООП упрощает разработку больших программ, потому что код становится более структурированным, повторно используемым и удобным для сопровождения.

4. Паттерны Singleton и Factory Method

Паттерны проектирования – это типовые решения часто встречающихся задач в программировании.

Singleton – паттерн, который гарантирует, что у класса будет только один объект. Обычно он используется для логирования, работы с настройками или объектами, которые должны существовать в единственном экземпляре.

Пример Singleton:

```

1    class Logger {
2        private static Logger instance;
3
4        private Logger() {
5        }
6
7        public static Logger getInstance() {

```

java

```

8      if (instance == null) {
9          instance = new Logger();
10     }
11
12     return instance;
13 }
14
15 public void log(String message) {
16     System.out.println(message);
17 }
18 }

```

Использование:

```

1  Logger logger = Logger.getInstance();
2  logger.log("Программа запущена");

```

java

Конструктор сделан private, поэтому объект нельзя создать через new. Получить объект можно только через метод getInstance().

Factory Method – паттерн, который выносит создание объектов в отдельный метод или класс. Он полезен, когда программа должна создавать разные объекты в зависимости от условий.

Пример:

```

1  interface Transport {
2      void move();
3  }
4
5  class Car implements Transport {
6      public void move() {
7          System.out.println("Машина едет");
8      }
9  }
10

```

java

```

11  class Bike implements Transport {
12      public void move() {
13          System.out.println("Велосипед едет");
14      }
15  }
16
17  class TransportFactory {
18      public Transport create(String type) {
19          if (type.equals("car")) {
20              return new Car();
21          }
22
23          if (type.equals("bike")) {
24              return new Bike();
25          }
26
27          throw new IllegalArgumentException("Неизвестный тип");
28      }
29  }

```

Factory Method уменьшает зависимость кода от конкретных классов и делает программу гибче.

5. Коллекции Java

Коллекции в Java используются для хранения групп объектов. Они находятся в пакете `java.util`.

Основные виды коллекций:

- List – список, который хранит элементы в порядке добавления и допускает дубликаты;
- Set – множество, которое хранит только уникальные элементы;
- Map – структура, которая хранит пары ключ-значение.

Пример List:

```

1 List<String> names = new ArrayList<>();
2
3 names.add("Анна");
4 names.add("Иван");
5 names.add("Анна");
6
7 System.out.println(names);

```

java

В List можно хранить одинаковые значения.

Пример Set:

```

1 Set<String> cities = new HashSet<>();
2
3 cities.add("Москва");
4 cities.add("Казань");
5 cities.add("Москва");
6
7 System.out.println(cities);

```

java

В Set повторяющееся значение не будет добавлено второй раз.

Пример Map:

```

1 Map<Integer, String> users = new HashMap<>();
2
3 users.put(1, "Анна");
4 users.put(2, "Иван");
5
6 System.out.println(users.get(1));

```

java

Для перебора коллекций можно использовать обычный цикл, расширенный for, forEach или Iterator.

Пример перебора:

```

1 for (String name : names) {
2     System.out.println(name);

```

java

```
3  }  
4  
5  users.forEach((id, name) -> {  
6      System.out.println(id + ": " + name);  
7  });
```

Коллекции часто используются при работе с базами данных, REST API, списками пользователей, товарами, заказами и другими наборами данных.

6. Исключения в Java

Исключения используются для обработки ошибок, которые возникают во время выполнения программы. Например, ошибка может появиться при делении на ноль, чтении несуществующего файла или неправильном вводе пользователя.

Конструкция try-catch позволяет перехватить ошибку и обработать её без аварийного завершения программы.

Пример:

```
1  try {  
2      int result = 10 / 0;  
3      System.out.println(result);  
4  } catch (ArithmeticException e) {  
5      System.out.println("Ошибка деления на ноль");  
6  }
```

java

Блок try содержит код, в котором может возникнуть ошибка. Блок catch выполняется, если исключение действительно произошло.

Также можно использовать блок finally. Он выполняется всегда, независимо от того, была ошибка или нет.

```
1  try {  
2      System.out.println("Работа с ресурсом");  
3  } catch (Exception e) {  
4      System.out.println("Ошибка");  
5  } finally {  
6      System.out.println("Завершение работы");
```

java

```
7 }
```

В Java можно создавать собственные исключения. Это удобно, когда нужно описать ошибку предметной области приложения.

Пример собственного исключения:

```
1 class InvalidAgeException extends Exception {
2     public InvalidAgeException(String message) {
3         super(message);
4     }
5 }
```

java

Использование:

```
1 public void checkAge(int age) throws InvalidAgeException {
2     if (age < 18) {
3         throw new InvalidAgeException("Возраст меньше 18 лет");
4     }
5 }
```

java

Исключения делают программу надёжнее, потому что позволяют заранее продумать обработку ошибочных ситуаций.

7. Потоки ввода-вывода

Потоки ввода-вывода используются для чтения и записи данных. В Java для работы с файлами часто применяются `BufferedReader` и `BufferedWriter`.

`BufferedReader` позволяет читать текст из файла построчно. Это удобно, если файл содержит несколько строк данных.

Пример чтения файла:

```
1 try (BufferedReader reader =
2     new BufferedReader(new FileReader("input.txt"))) {
3
4     String line;
5
6     while ((line = reader.readLine()) != null) {
```

java

```

7      System.out.println(line);
8  }
9
10 } catch (IOException e) {
11     System.out.println("Ошибка чтения файла");
12 }

```

BufferedWriter используется для записи текста в файл.

Пример записи:

```

1  try (BufferedWriter writer =
2      new BufferedWriter(new FileWriter("output.txt"))) {
3
4      writer.write("Привет, Java!");
5      writer.newLine();
6      writer.write("Запись в файл выполнена");
7
8  } catch (IOException e) {
9      System.out.println("Ошибка записи файла");
10 }

```

java

Конструкция try-with-resources автоматически закрывает файл после завершения работы. Это лучше, чем вручную вызывать close().

Потоки ввода-вывода применяются для сохранения настроек, логов, отчётов, текстовых файлов и других данных.

8. Многопоточность

Многопоточность позволяет выполнять несколько частей программы одновременно. Поток – это отдельная последовательность выполнения внутри программы.

В Java поток можно создать двумя основными способами:

1. Наследоваться от класса Thread.
2. Реализовать интерфейс Runnable.

Пример через Thread:

```

1  class MyThread extends Thread {
2      public void run() {
3          System.out.println("Поток запущен");
4      }
5  }
6
7  MyThread thread = new MyThread();
8  thread.start();

```

java

Метод start() запускает новый поток. Метод run() содержит код, который будет выполняться в этом потоке.

Пример через Runnable:

```

1  Runnable task = () -> {
2      System.out.println("Задача выполняется");
3  };
4
5  Thread thread = new Thread(task);
6  thread.start();

```

java

Если несколько потоков работают с общими данными, может возникнуть конфликт. Для защиты данных используется синхронизация.

Пример:

```

1  class Counter {
2      private int value = 0;
3
4      public synchronized void increment() {
5          value++;
6      }
7
8      public int getValue() {
9          return value;
10     }

```

java

Ключевое слово `synchronized` не даёт нескольким потокам одновременно выполнять один и тот же критический участок кода.

Многопоточность применяется в серверных приложениях, обработке запросов, фоновых задачах и асинхронных вычислениях.

9. Основы Spring Boot

Spring Boot – это фреймворк, который упрощает создание приложений на Java. Он основан на Spring Framework, но избавляет разработчика от большого количества ручной настройки.

Главная аннотация Spring Boot-приложения:

```
1 @SpringBootApplication
```

`java`

Эта аннотация объединяет несколько возможностей:

- автоматическую конфигурацию;
- поиск компонентов приложения;
- запуск Spring-контейнера.

Пример главного класса:

```
1 @SpringBootApplication
2 public class App {
3     public static void main(String[] args) {
4         SpringApplication.run(App.class, args);
5     }
6 }
```

`java`

Spring Boot сам настраивает встроенный сервер, например Tomcat, поэтому приложение можно запустить как обычную Java-программу.

Аннотация `@RestController` используется для создания REST-контроллера.

Пример:

```
1 @RestController
2 public class HelloController {
3
```

`java`

```
4 @GetMapping("/hello")
5 public String hello() {
6     return "Hello";
7 }
8 }
```

После запуска приложения endpoint /hello будет доступен через браузер или Postman.

Spring Boot часто используется для создания REST API, веб-приложений, микросервисов и серверной части информационных систем.

10. REST API в Spring Boot

REST API – это способ взаимодействия клиента и сервера через HTTP-запросы. Клиент отправляет запрос, а сервер возвращает ответ, чаще всего в формате JSON.

Основные HTTP-методы:

- GET – получение данных;
- POST – создание данных;
- PUT – полное обновление данных;
- PATCH – частичное обновление;
- DELETE – удаление данных.

Пример GET-запроса:

```
1 @RestController
2 @RequestMapping("/users")
3 public class UserController {
4
5     @GetMapping
6     public List<String> getUsers() {
7         return List.of("Анна", "Иван");
8     }
9 }
```

java

Если открыть /users, сервер вернёт список пользователей.

Пример POST-запроса:

```
1 @PostMapping
2 public String createUser(@RequestBody String name) {
3     return "Создан пользователь: " + name;
4 }
```

java

Аннотация `@RequestBody` говорит Spring, что данные нужно взять из тела запроса. Обычно в теле запроса передаётся JSON.

Пример JSON:

```
1 {
2     "name": "Анна",
3     "age": 20
4 }
```

json

REST API применяется для связи фронтенда с сервером, мобильных приложений с базой данных, а также для интеграции разных систем.

11. Dependency Injection

Dependency Injection – это внедрение зависимостей. Суть в том, что объект не создаёт нужные ему зависимости самостоятельно, а получает их извне.

В Spring Boot объекты создаёт и хранит Spring-контейнер. Такие объекты называются бинами. Контейнер сам определяет, какие классы нужны друг другу, и внедряет зависимости.

Пример сервиса:

```
1 @Service
2 public class UserService {
3
4     public String getMessage() {
5         return "Список пользователей";
6     }
7 }
```

java

Пример внедрения через `@Autowired`:

```

1  @RestController
2  public class UserController {
3
4      @Autowired
5      private UserService userService;
6
7      @GetMapping("/message")
8      public String message() {
9          return userService.getMessage();
10     }
11 }

```

Spring сам создаст объект UserService и передаст его в контроллер.

Также зависимости можно внедрять через конструктор:

```

1  @RestController
2  public class UserController {
3
4      private final UserService userService;
5
6      public UserController(UserService userService) {
7          this.userService = userService;
8      }
9  }

```

Внедрение через конструктор считается более удобным и безопасным, потому что сразу видно, какие зависимости нужны классу.

Dependency Injection уменьшает связанность кода, упрощает тестирование и делает приложение более гибким.

12. Работа с базой данных через Spring Data JPA

Spring Data JPA используется для работы с базой данных через Java-объекты. Вместо того чтобы вручную писать SQL-запросы для простых операций, разработчик создаёт сущности и репозитории.

Сущность – это Java-класс, который соответствует таблице в базе данных. Для обозначения сущности используется аннотация @Entity.

Пример сущности:

```
1  @Entity java
2  public class User {
3
4      @Id
5      @GeneratedValue(strategy = GenerationType.IDENTITY)
6      private Long id;
7
8      private String name;
9      private String email;
10 }
```

Аннотация @Id показывает первичный ключ. Аннотация @GeneratedValue указывает, что значение ключа будет генерироваться автоматически.

Репозиторий используется для выполнения операций с базой данных.

```
1  public interface UserRepository java
2      extends JpaRepository<User, Long> {
3  }
```

После создания такого интерфейса становятся доступны методы:

- findAll() – получить все записи;
- findById() – найти по id;
- save() – сохранить или обновить объект;
- delete() – удалить объект;
- count() – посчитать количество записей.

Пример использования:

```
1  @Service java
2  public class UserService {
3
4      private final UserRepository repository;
```

```

5
6     public UserService(UserRepository repository) {
7         this.repository = repository;
8     }
9
10    public List<User> getAll() {
11        return repository.findAll();
12    }
13 }

```

Spring Data JPA удобен тем, что позволяет быстро создавать CRUD-функциональность и связывать приложение с базой данных.

13. Тестирование с JUnit

JUnit – это библиотека для unit-тестирования Java-кода. Unit-тест проверяет небольшую часть программы, например один метод.

Главная цель тестирования – убедиться, что код работает правильно и не ломается после изменений.

Пример класса:

```

1     public class Calculator {
2
3         public int sum(int a, int b) {
4             return a + b;
5         }
6     }

```

java

Пример теста:

```

1     import org.junit.jupiter.api.Test;
2
3     import static org.junit.jupiter.api.Assertions.assertEquals;
4
5     class CalculatorTest {

```

java

```

6
7     @Test
8     void testSum() {
9         Calculator calculator = new Calculator();
10
11         int result = calculator.sum(2, 3);
12
13         assertEquals(5, result);
14     }
15 }

```

Аннотация `@Test` показывает, что метод является тестом. Метод `assertEquals()` сравнивает ожидаемый и фактический результат.

Если результат совпал, тест проходит успешно. Если нет – тест завершается ошибкой.

Unit-тесты помогают быстрее находить ошибки и повышают надёжность программы.

14. Сериализация объектов в JSON

Сериализация – это преобразование объекта в другой формат, например в JSON. Десериализация – обратный процесс, когда JSON преобразуется обратно в объект.

JSON часто используется в REST API, потому что он удобен для обмена данными между клиентом и сервером.

Пример класса:

```

1     public class User {
2         private String name;
3         private int age;
4
5         public User() {
6             }
7

```

```
8     public User(String name, int age) {
9         this.name = name;
10        this.age = age;
11    }
12 }
```

Пример сериализации с Jackson:

```
1  ObjectMapper mapper = new ObjectMapper();
2
3  User user = new User("Анна", 20);
4
5  String json = mapper.writeValueAsString(user);
6
7  System.out.println(json);
```

java

Пример результата:

```
1  {
2    "name": "Анна",
3    "age": 20
4  }
```

json

Пример десериализации:

```
1  String json = "{\"name\":\"Анна\",\"age\":20}";
2
3  User user = mapper.readValue(json, User.class);
```

java

В Spring Boot сериализация и десериализация часто выполняются автоматически при работе с `@RequestBody` и ответами REST-контроллеров.

15. Основы безопасности

Безопасность приложения нужна для защиты данных пользователя и предотвращения неправильного доступа к функциям системы.

Основные меры безопасности:

- проверка входных данных;
- ограничение доступа к защищённым endpoint'ам;
- хранение паролей в виде хеша;
- защита от SQL-инъекций;
- разграничение ролей пользователей;
- использование HTTPS в реальных проектах.

Проверка данных пользователя нужна для того, чтобы в систему не попадали некорректные или опасные значения.

Пример простой проверки:

```
1 public void register(String username, String password) {  
2     if (username == null || username.isBlank()) {  
3         throw new IllegalArgumentException("Логин не может быть пустым");  
4     }  
5  
6     if (password.length() < 6) {  
7         throw new IllegalArgumentException("Пароль слишком короткий");  
8     }  
9 }
```

Базовая авторизация использует логин и пароль. Пользователь передаёт данные для входа, а сервер проверяет их и решает, можно ли дать доступ к ресурсу.

В Spring Security можно ограничить доступ к endpoint'ам, например разрешить один адрес всем, а другой только авторизованным пользователям.

Безопасность особенно важна в приложениях, где есть личные данные, платежи, роли пользователей и административные функции.

16. Асинхронные методы

Асинхронные методы позволяют выполнять задачу в отдельном потоке, не блокируя основной поток программы. Это удобно, если операция выполняется долго: отправка письма, обработка файла, расчёт отчёта или обращение к внешнему сервису.

В Spring Boot для асинхронных методов используется аннотация `@Async`.

Чтобы включить поддержку асинхронности, нужно добавить `@EnableAsync`.

Пример:

```
1 @Configuration
```

```
java
```

```
2 @EnableAsync
```

```
3 public class AsyncConfig {
```

```
4 }
```

Асинхронный метод:

```
1 @Service
```

```
java
```

```
2 public class NotificationService {
```

```
3
```

```
4     @Async
```

```
5     public void sendMessage() throws InterruptedException {
```

```
6         Thread.sleep(2000);
```

```
7         System.out.println("Сообщение отправлено");
```

```
8     }
```

```
9 }
```

Вызов метода:

```
1 @RestController
```

```
java
```

```
2 public class NotificationController {
```

```
3
```

```
4     private final NotificationService service;
```

```
5
```

```
6     public NotificationController(NotificationService service) {
```

```
7         this.service = service;
```

```
8     }
```

```
9
```

```
10    @GetMapping("/send")
```

```
11    public String send() throws InterruptedException {
```

```
12        service.sendMessage();
```

```
13        return "Запрос принят";
```

```
14    }
```

```
15 }
```

В этом примере клиент сразу получит ответ, а отправка сообщения выполнится отдельно.

Асинхронность полезна для повышения отзывчивости приложения, но её нужно использовать аккуратно, особенно если метод работает с общими данными.

17. Планирование задач

Планирование задач нужно для автоматического выполнения методов по расписанию. В Spring Boot для этого используется аннотация `@Scheduled`.

Чтобы включить планировщик, нужно добавить `@EnableScheduling`.

Пример:

```
1 @Configuration
2 @EnableScheduling
3 public class ScheduleConfig {
4 }
```

java

Пример задачи:

```
1 @Component
2 public class ReportTask {
3
4     @Scheduled(fixedRate = 5000)
5     public void printMessage() {
6         System.out.println("Задача выполняется каждые 5 секунд");
7     }
8 }
```

java

Параметр `fixedRate = 5000` означает, что метод будет запускаться каждые 5000 миллисекунд.

Также можно использовать `cron`.

Пример:

```
1 @Scheduled(cron = "0 0 12 * * *")
2 public void runEveryDay() {
3     System.out.println("Задача выполняется каждый день в 12:00");
4 }
```

java

Планировщик задач применяется для регулярной очистки данных, формирования отчётов, проверки статусов, отправки уведомлений и других повторяющихся действий.

18. Конфигурация приложения

Конфигурация Spring Boot-приложения обычно хранится в файле `application.properties`. В нём задают порт сервера, имя приложения, настройки базы данных, логирования и другие параметры.

Пример:

```
server.port=8081  
spring.application.name=demo-app
```

```
app.owner=Student
```

Порт 8081 означает, что приложение будет запускаться не на стандартном 8080, а на другом порту.

Получить значение из конфигурации можно через аннотацию `@Value`.

Пример:

```
1  @Component  
2  public class AppInfo {  
3  
4      @Value("${spring.application.name}")  
5      private String appName;  
6  
7      @Value("${app.owner}")  
8      private String owner;  
9  
10     public void printInfo() {  
11         System.out.println(appName + " " + owner);  
12     }  
13 }
```

java

Также в `application.properties` часто прописывают подключение к базе данных:

```
spring.datasource.url=jdbc:postgresql://localhost:5432/demo
spring.datasource.username=postgres
spring.datasource.password=1234
```

Вынос настроек в отдельный файл удобен тем, что можно менять параметры приложения без изменения Java-кода.

19. MVC-паттерн

MVC – это архитектурный паттерн, который разделяет приложение на три части: Model, View и Controller.

Model отвечает за данные и бизнес-логику. Например, это могут быть классы User, Order, Product.

View отвечает за отображение данных пользователю. В Spring MVC для этого часто используется Thymeleaf.

Controller принимает запросы пользователя, вызывает нужные сервисы и передаёт данные во View.

Пример модели:

```
1 public class User {
2     private String name;
3
4     public User(String name) {
5         this.name = name;
6     }
7 }
```

java

Пример контроллера:

```
1 @Controller
2 public class PageController {
3
4     @GetMapping("/profile")
5     public String profile(Model model) {
6         model.addAttribute("name", "Анна");
7         return "profile";
8     }
9 }
```

java

```
8    }  
9  }
```

Пример представления profile.html:

```
<h1 th:text="${name}"></h1>
```

MVC делает приложение более понятным: контроллер не отвечает за хранение данных, модель не занимается отображением, а представление не содержит основную бизнес-логику.

Этот подход часто используется в веб-приложениях.

20. Swagger

Swagger – это инструмент для документирования и тестирования REST API. В современных Spring Boot-проектах обычно используется OpenAPI через библиотеку springdoc-openapi.

Swagger UI показывает список endpoint'ов, методы запросов, параметры, структуры JSON и возможные ответы сервера.

Основные преимущества Swagger:

- автоматическая генерация документации;
- возможность тестировать API прямо из браузера;
- удобство для разработчиков фронтенда;
- наглядное описание входных и выходных данных.

Пример зависимости для Maven:

```
<dependency>  
  <groupId>org.springdoc</groupId>  
  <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>  
  <version>2.5.0</version>  
</dependency>
```

Пример контроллера:

```
1  @RestController  
2  @RequestMapping("/users")  
3  public class UserController {  
4  
5      @GetMapping
```

java

```
6 public List<String> getUsers() {  
7     return List.of("Анна", "Иван");  
8 }  
9 }
```

После подключения Swagger UI обычно доступен по адресу:

/swagger-ui/index.html

Swagger особенно полезен при разработке REST API, потому что документация обновляется вместе с кодом.

21. Аннотации в Java

Аннотации – это специальные метки, которые добавляют к коду метаданные. Они могут применяться к классам, методам, полям, параметрам и другим элементам.

Аннотации не всегда напрямую изменяют поведение кода, но они могут использоваться компилятором, фреймворками или инструментами анализа.

Примеры стандартных аннотаций:

- `@Override` – показывает, что метод переопределяет метод родительского класса;
- `@Deprecated` – показывает, что элемент устарел;
- `@SuppressWarnings` – подавляет предупреждения компилятора.

Пример `@Override`:

```
1 class User {  
2  
3     @Override  
4     public String toString() {  
5         return "User";  
6     }  
7 }
```

java

Пример `@Deprecated`:

```
1 @Deprecated
```

java

```
2 public void oldMethod() {  
3     System.out.println("Старый метод");  
4 }
```

В Spring аннотации используются очень активно:

- @Component;
- @Service;
- @Repository;
- @Controller;
- @RestController;
- @Autowired;
- @Entity.

Можно создавать собственные аннотации.

Пример:

```
1 public @interface MyAnnotation {  
2     String value();  
3 }
```

java

Аннотации делают код более декларативным: разработчик описывает, что нужно сделать, а фреймворк выполняет это автоматически.

22. Лямбда-выражения и функциональные интерфейсы

Лямбда-выражения позволяют передавать поведение как значение. Они делают код короче и удобнее, особенно при работе с коллекциями и Stream API.

Лямбда имеет общий вид:

```
1 (параметры) -> действие
```

java

Пример:

```
1 List<String> names = List.of("Анна", "Иван");  
2  
3 names.forEach(name -> System.out.println(name));
```

java

Функциональный интерфейс – это интерфейс, у которого есть только один абстрактный метод.

Пример:

```
1 @FunctionalInterface
2 interface Operation {
3     int apply(int a, int b);
4 }
```

java

Использование:

```
1 Operation sum = (a, b) -> a + b;
2
3 int result = sum.apply(2, 3);
```

java

Java уже содержит готовые функциональные интерфейсы:

- Predicate<T> – проверяет условие;
- Function<T, R> – преобразует значение;
- Consumer<T> – принимает значение и ничего не возвращает;
- Supplier<T> – возвращает значение.

Пример Predicate:

```
1 Predicate<Integer> isEven = x -> x % 2 == 0;
2
3 System.out.println(isEven.test(4));
```

java

Лямбда-выражения часто используются вместе со Streams API, коллекциями и обработчиками событий.

23. Streams API

Streams API используется для обработки коллекций в функциональном стиле. Stream – это поток данных, над которым можно выполнять операции фильтрации, преобразования, сортировки и сбора результата.

Важно понимать, что Stream обычно не изменяет исходную коллекцию, а формирует новый результат.

Основные операции:

- filter – фильтрация элементов;
- map – преобразование элементов;
- sorted – сортировка;
- limit – ограничение количества;
- reduce – свёртка значений;
- collect – сбор результата в коллекцию.

Пример фильтрации:

```
1 List<Integer> numbers = List.of(1, 2, 3, 4, 5, 6);
2
3 List<Integer> even = numbers.stream()
4     .filter(x -> x % 2 == 0)
5     .toList();
6
7 System.out.println(even);
```

java

Результат:

[2, 4, 6]

Пример преобразования:

```
1 List<String> names = List.of("anna", "ivan");
2
3 List<String> upperNames = names.stream()
4     .map(String::toUpperCase)
5     .toList();
```

java

Пример суммы:

```
1 int sum = numbers.stream()
2     .reduce(0, Integer::sum);
```

java

Streams API делает код короче и понятнее, если нужно выполнить несколько операций над коллекцией.

Например: получить список имён пользователей старше 18 лет, отсортировать их и вернуть клиенту через REST API.

24. Generics в Java

Generics – это обобщения. Они позволяют создавать классы, интерфейсы и методы, которые работают с разными типами данных, но при этом сохраняют безопасность типов.

Без Generics пришлось бы часто использовать тип `Object`, а затем вручную приводить типы. Это неудобно и может привести к ошибкам.

Пример обобщённого класса:

```
1  class Box<T> {
2      private T value;
3
4      public void setValue(T value) {
5          this.value = value;
6      }
7
8      public T getValue() {
9          return value;
10     }
11 }
```

java

Использование:

```
1  Box<String> stringBox = new Box<>();
2  stringBox.setValue("Java");
3
4  String value = stringBox.getValue();
```

java

Здесь `T` заменяется на `String`.

Generics активно используются в коллекциях:

```
1  List<String> names = new ArrayList<>();
2  Map<Integer, String> users = new HashMap<>();
```

java

Можно ограничить тип.

Пример:

```
1 class NumberBox<T extends Number> {  
2     private T value;  
3 }
```

java

T extends Number означает, что вместо T можно использовать только Number или его наследников, например Integer, Double, Long.

Generics повышают безопасность кода и позволяют находить ошибки ещё на этапе компиляции.

25. Работа с датой и временем

В современных версиях Java для работы с датой и временем используются классы из пакета `java.time`.

Основные классы:

- `LocalDate` – дата без времени;
- `LocalTime` – время без даты;
- `LocalDateTime` – дата и время;
- `DateTimeFormatter` – форматирование даты и времени.

Пример текущей даты:

```
1 LocalDate date = LocalDate.now();  
2  
3 System.out.println(date);
```

java

Пример текущего времени:

```
1 LocalTime time = LocalTime.now();  
2  
3 System.out.println(time);
```

java

Пример даты и времени:

```
1 LocalDateTime dateTime = LocalDateTime.now();  
2  
3 System.out.println(dateTime);
```

java

Создание конкретной даты:

```
1 LocalDate examDate = LocalDate.of(2026, 5, 20);
```

java

Форматирование:

```
1 DateTimeFormatter formatter =  
2     DateTimeFormatter.ofPattern("dd.MM.yyyy");  
3  
4 String text = examDate.format(formatter);
```

java

Пример результата:

20.05.2026

Работа с датой и временем нужна в расписаниях, заказах, логах, отчётах, бронировании и других системах.

26. Паттерн Builder

Builder – это паттерн проектирования, который используется для пошагового создания сложных объектов.

Он полезен, когда у класса много полей, и не хочется создавать огромный конструктор с большим количеством параметров.

Проблема обычного конструктора:

```
1 User user = new User("Анна", 20, "anna@mail.ru", true);
```

java

По такому коду не всегда понятно, что означает каждый аргумент.

Пример Builder:

```
1 class User {  
2     private String name;  
3     private int age;  
4     private String email;  
5  
6     private User(Builder builder) {  
7         this.name = builder.name;  
8         this.age = builder.age;  
9         this.email = builder.email;
```

java

```

10     }
11
12     public static class Builder {
13         private String name;
14         private int age;
15         private String email;
16
17         public Builder name(String name) {
18             this.name = name;
19             return this;
20         }
21
22         public Builder age(int age) {
23             this.age = age;
24             return this;
25         }
26
27         public Builder email(String email) {
28             this.email = email;
29             return this;
30         }
31
32         public User build() {
33             return new User(this);
34         }
35     }
36 }

```

Использование:

```

1  User user = new User.Builder()
2      .name("Анна")
3      .age(20)

```

java

```
4     .email("anna@mail.ru")
5     .build();
```

Такой код лучше читается, потому что видно, какие значения каким полям соответствуют.

Builder часто применяется для DTO, конфигурационных объектов и классов с большим количеством параметров.

27. Паттерн Observer

Observer – это паттерн проектирования, который используется для уведомления одних объектов об изменениях в другом объекте.

Есть объект-источник, за которым наблюдают, и есть наблюдатели. Когда состояние источника меняется, он уведомляет всех наблюдателей.

Пример интерфейса наблюдателя:

```
1 interface Observer {
2     void update(String message);
3 }
```

java

Пример наблюдателя:

```
1 class EmailObserver implements Observer {
2     public void update(String message) {
3         System.out.println("Email: " + message);
4     }
5 }
```

java

Пример источника событий:

```
1 class NewsPublisher {
2     private List<Observer> observers = new ArrayList<>();
3
4     public void addObserver(Observer observer) {
5         observers.add(observer);
6     }
7 }
```

java

```

8     public void notifyObservers(String news) {
9         for (Observer observer : observers) {
10             observer.update(news);
11         }
12     }
13 }

```

Использование:

```

1  NewsPublisher publisher = new NewsPublisher();
2
3  publisher.addObserver(new EmailObserver());
4  publisher.notifyObservers("Новая новость");

```

java

Observer применяется в системах уведомлений, графических интерфейсах, подписках, событиях и реактивном программировании.

28. Кэширование и оптимизация

Кэширование – это сохранение результата операции для повторного использования. Если метод часто возвращает одни и те же данные, результат можно сохранить в кэше и не выполнять тяжёлую операцию каждый раз заново.

В Spring для кэширования используется Spring Cache.

Чтобы включить кэширование, добавляют аннотацию `@EnableCaching`.

Пример:

```

1  @Configuration
2  @EnableCaching
3  public class CacheConfig {
4  }

```

java

Пример метода с кэшем:

```

1  @Service
2  public class UserService {
3

```

java

```
4  @Cacheable("users")
5  public List<String> getUsers() {
6      System.out.println("Загрузка пользователей");
7      return List.of("Анна", "Иван");
8  }
9  }
```

Аннотация `@Cacheable` означает, что результат метода будет сохранён. При следующем вызове с теми же параметрами Spring может вернуть данные из кэша.

Кэширование уменьшает нагрузку на базу данных и ускоряет работу приложения.

Но кэш нужно использовать аккуратно. Если данные часто меняются, можно получить устаревший результат. Для очистки кэша используются аннотации `@CacheEvict` и `@CachePut`.

Пример:

```
1  @CacheEvict(value = "users", allEntries = true)
2  public void clearCache() {
3  }
```

java

Кэширование применяют для справочников, редко изменяемых данных, настроек и тяжёлых вычислений.

29. Транзакции в Spring

Транзакция – это группа операций, которая должна выполняться полностью или не выполняться вообще.

Например, при оплате заказа нужно:

1. Списать деньги.
2. Создать запись об оплате.
3. Изменить статус заказа.

Если одна из операций завершится ошибкой, остальные изменения нужно отменить. Для этого используются транзакции.

В Spring транзакции обычно задаются аннотацией `@Transactional`.

Пример:

```
1  @Service java
2  public class PaymentService {
3
4      @Transactional
5      public void pay(Long orderId) {
6          savePayment(orderId);
7          updateOrderStatus(orderId);
8      }
9
10     private void savePayment(Long orderId) {
11         System.out.println("Оплата сохранена");
12     }
13
14     private void updateOrderStatus(Long orderId) {
15         System.out.println("Статус заказа изменён");
16     }
17 }
```

Если внутри метода `pay()` произойдёт ошибка, Spring откатит изменения.

Основные свойства транзакции:

- атомарность – операции выполняются как единое целое;
- согласованность – данные остаются корректными;
- изолированность – параллельные транзакции не должны мешать друг другу;
- долговечность – после фиксации данные сохраняются.

Аннотация `@Transactional` часто используется в сервисном слое, где выполняется бизнес-логика.

30. Документирование REST API

Документирование REST API нужно для того, чтобы разработчики понимали, какие запросы поддерживает сервер, какие параметры нужно передавать и какие ответы можно получить.

В Spring Boot для этого часто используют Swagger/OpenAPI.

Документация API обычно содержит:

- список endpoint'ов;
- HTTP-методы;
- параметры запроса;
- структуру JSON;
- коды ответов;
- описание ошибок.

Пример REST-контроллера:

```
1  @RestController
2  @RequestMapping("/products")
3  public class ProductController {
4
5      @GetMapping
6      public List<String> getProducts() {
7          return List.of("Ноутбук", "Телефон");
8      }
9
10     @PostMapping
11     public String createProduct(@RequestBody String name) {
12         return "Товар создан: " + name;
13     }
14 }
```

java

Swagger автоматически найдёт эти методы и покажет их в интерфейсе документации.

Swagger UI удобен тем, что можно не только читать описание API, но и отправлять тестовые запросы прямо из браузера.

Документирование особенно важно, если над проектом работает несколько человек: backend-разработчики, frontend-разработчики, тестировщики и аналитики.

OpenAPI также можно использовать для генерации клиентского кода и схем API.